LANGCHAIN MASTERY SERIES

# Day 3: Output Parsers & LCEL Pipelines

Converting Raw Model Text into Structured Python Dictionaries & Type-Safe Objects

## 1. Understanding Output Parsers

By default, LLMs return responses as unstructured text strings. Production applications require reliable, structured data format (such as JSON or validated Python objects) for database storage and downstream API integration. **Output Parsers** automate formatting instructions, validate model output, and convert response text into native data structures.

## 2. LCEL & The Pipe Operator (|)

LangChain Expression Language (LCEL) links components sequentially using the | pipe operator, creating an automated assembly pipeline:




```
chain = prompt_with_instructions | llm | parser
```

## 3. Standard Parsers Covered

| Parser Type | Primary Purpose | Output Format |
|---|---|---|
| **JsonOutputParser** | Formats output into valid JSON structure automatically. | Python `dict` |
| **PydanticOutputParser** | Enforces strict data types, data validation, and dot-notation access. | Pydantic Object |

## 4. Production Implementation: Pydantic Data Model

```python
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import PydanticOutputParser
from pydantic import BaseModel, Field

load_dotenv()

# 1. Define Data Schema with Types & Descriptions
class FitnessPlan(BaseModel):
    goal: str = Field(description="The fitness goal of the user")
    duration_days: int = Field(description="Number of days for the plan")
    workout_summary: str = Field(description="Brief workout routine overview")
    daily_calories: int = Field(description="Recommended daily calorie intake")

# 2. Setup Parser & Inject Format Instructions
parser = PydanticOutputParser(pydantic_object=FitnessPlan)
chat_template = ChatPromptTemplate.from_messages([
    ("system", "You are an expert coach. Return response according to
schema.\n{format_instructions}"),
    ("human", "Goal: {goal}, Level: {fitness_level}, Diet: {diet_preferences}")
])

prompt = chat_template.partial(format_instructions=parser.get_format_instructions())

# 3. Construct LCEL Chain & Invoke
chain = prompt | ChatOpenAI(model="gpt-4o-mini", temperature=0.7) | parser
result = chain.invoke({"goal": "Weight Loss", "fitness_level": "Beginner",
"diet_preferences": "Desi food"})

# Access attributes directly via dot notation
print(f"Goal: {result.goal} | Target Calories: {result.daily_calories}")
```

**Key Milestones Reached in Day 3:**

- Automated instruction formatting using `parser.get_format_instructions()`.
- Eliminated manual parsing logic (`json.loads`) via unified LCEL chains.
- Achieved full type-safety (numeric outputs guaranteed as integers rather than strings).